CASE-STUDY PRACTICE | CAPSTONE 4 | STUDY EDITION

Search Ranking: From Boosted Trees to a Neural Ranker

Scenario questions on ranking listings for a marketplace, each opening into deeper follow-ups, with answers that explain how to think. Built on the AI System Design Primer case study of the same name.

Jump to the contents | The case in one page | Numbers to know | Decision table | Sources

HOW TO STUDY WITH THIS BOOK

1. Read the amber **concepts box** at the start of a part and look up anything unfamiliar. 2. Read a **scenario** and, before reading further, write down what you would ask and what you would check. 3. Read the green **how to think** box, which names the mental move the question tests. 4. Walk down the nested steps: cover each answer, predict it, then compare. Each answer exposes the next question. The last step of every question asks for the general lesson in one sentence.

The pattern behind most questions

- A number or a technique arrives with authority: a published benchmark, a cost estimate, a shard count, a rate.
- The interviewer asks what it assumes, and what happens when the assumption fails.
- A strong answer names the hidden assumption, puts a number on it, and says how to measure it on your own data.
- The last step usually reframes the original question.

EVIDENCE TAGS

source-confirmed: checked against the named primary source. **illustrative**: this document's planning assumption. **recomputed**: arithmetic re-derived from the stated inputs. **reasoned**: an argument from the design, not tested.

A NOTE ON SOURCES AND ON WHAT WAS CHECKED

The scenarios are written for this document; none comes from a real company question bank. Nothing here was run on data, a model or a serving system: figures are the paper's or arithmetic from stated inputs, and candidate counts, record sizes and hardware speeds are illustrative. The last page lists the sources and says how much of each was read. A separate Mock Interview edition (answers hidden) uses the same questions and numbering.

Contents

Every line below is a link: click it to jump to that page. The outline panel of your PDF viewer lists the same entries, and the “Contents” link in every page footer brings you back here.

Front matter

A The case in one page 3

B Numbers to know 4

C Decision table 5

Parts at a glance

1 Framing the problem: labels, the metric, the plateau and the sizing 6

2 Features and data: normalization, smoothness and the ID that failed 10

3 The models: trees, LambdaRank, the hybrid and the deep network 15

4 Serving, evaluation and operations 21

Sources and confidence 26

Every question, by part

Part 1 Framing the problem: labels, the metric, the plateau and the sizing 6

01 What is the label, and who produces it? Framing 6

02 The model has stopped improving. What now? Plateau 7

03 Compute NDCG by hand Calculation 8

04 Where does the cost sit? Estimation 9

Part 2 Features and data: normalization, smoothness and the ID that failed 10

05 Why did the same features break the network? Normalization 10

06 Plot the distribution first Data quality 11

07 Embed every listing ID? Features 12

08 Review the price transform Code review 13

Part 3 The models: trees, LambdaRank, the hybrid and the deep network 15

09 Trees or a neural network? Decision 15

10 What does LambdaRank do to the loss? Algorithm 16

11 Combine three models, then drop two Architecture 17

12 Review the LambdaRank loss Code review 18

13 Read a learning curve Evaluation 19

Part 4 Serving, evaluation and operations 21

14 Feed the GPU Training pipeline 21

15 A static embedding: for training, not serving Pipeline 22

16 Score a TensorFlow model inside a Java server Serving 23

17 Trust the offline number? Evaluation 24

18 Which features matter, and what is not covered? Analysis 25

The case in one page

A summary of the case study to read before the questions.

Problem	Order the thousands of listings that match a search so that the one the guest would book comes first. A boosted-tree model improved bookings and then plateaued.
Core idea	Move to a neural ranker in stages, with a pairwise loss weighted by the change in NDCG, and treat it as a rethink of the whole system, not a drop-in model swap.
Timeline	Simple NN 04/2017, LambdaRank NN 06/2017, hybrid 03/2018, deep NN 06/2018 (source-confirmed, Figure 2 labels).
Label	The booked listing's impression has relevance 1, all others 0. Pairs come from the searches of guests who booked (source-confirmed).
Final model	195 inputs, then 127 and 83 ReLU units, 1.7 billion training pairs. Scaling the data 10 times replaced the hybrid's complexity (source-confirmed).
Failed attempts	A listing ID embedding overfit. The long-view multi-task model left bookings neutral. Hand-shaped noise gave about 1% offline NDCG and no significant online gain (source-confirmed).
First to break	The training input pipeline, then scoring in the Java stack. Protobuf and Dataset gave a 17 times speedup and GPU use near 90% from about 25% (source-confirmed).
Easy-to-miss traps	The static embedding is a training-speed trick, not a serving cache. Trees can be trained for NDCG too. 25% to 90% is 3.6 times, not 17. The NDCG discount is not a bias correction.

Numbers to know

Each figure carries the tag that says how it was obtained. The paper gives few numbers in its text, so candidate counts, record sizes and hardware speeds are illustrative.

Quantity	Value	Evidence
Paper's numbers	195 input features; 127 then 83 ReLU units; 1.7 billion pairs; 17 times speedup; GPU use about 25% to about 90%; about 1% offline NDCG from hand-shaped noise	source-confirmed (the paper)
Launch dates	Simple NN 04/2017; LambdaRank NN 06/2017; hybrid 03/2018; deep NN 06/2018; 14 months first to last	source-confirmed (Figure 2 labels); recomputed
Scoring cost	195 × 127 + 127 × 83 + 83 = 35,389 multiply-adds = 70,778 FLOPs; 2,000 candidates: 141.6 million FLOPs	illustrative; recomputed
Training compute	424,668 FLOPs per pair; 1.7 billion pairs: 7.2 × 10^14 FLOPs; about 144 s at 5 TFLOP/s	illustrative; recomputed
Input bytes	1.7 billion × 2 × 4 KB = 13.9 TB per pass; about 13,900 s at 1 GB/s, about 97 times the compute	illustrative; recomputed
Static table	2 million listings × 4 KB = 8.2 GB; each record read about 1,700 times per pass without it	illustrative; recomputed
Utilization against speedup	90 ÷ 25 = 3.6 times; 17 times would imply a start at 5.3%	recomputed
NDCG discounts	Positions 1, 2, 9, 10: 1.000, 0.631, 0.301, 0.289; gains 0.369 and 0.012 (31 times)	recomputed
LambdaRank pair loss	Weight 0.369 × base loss 0.474 (score gap 0.5) = 0.175; for ranks 10 and 9: weight 0.012	recomputed
Price transform	Median 120: price 60 gives −0.685; 300 gives 0.911; 3,000 gives 3.211; a monthly 3,600 gives 3.393	illustrative; recomputed
Listing bookings	At most 365 a year; 730 over two years; about 60 positives at 30 a year	source-confirmed (365); illustrative
Call overhead	2,000 × 195 × 4 B = 1.56 MB per query; 1.56 GB/s at 1,000 queries/s	illustrative; recomputed

Decision table

What this design chooses, and when to choose otherwise.

Decision	Default here	Choose otherwise when	Evidence
Model family	Neural ranker after the plateau	A tree model for a focused, medium-sized problem	source-confirmed
Training loss	Pairwise, weighted by change in NDCG	Pointwise regression for a first working pipeline	source-confirmed
Stages	Simple, then LambdaRank, then hybrid, then deep	Directly to the deep model if the system is ready (the paper leaves it open)	source-confirmed
Static features in training	GPU-resident table indexed by listing ID	Per-example features if they change quickly	source-confirmed
Listing identity	No per-listing embedding	An embedding only if each ID has enough data	source-confirmed
Input data format	Protobuf and Dataset	CSV only for a small prototype	source-confirmed
Scoring	A custom Java library	A model server if its call cost fits the budget	source-confirmed; reasoned
Ship decision	A/B test on bookings	Offline NDCG alone to choose what to test	source-confirmed

PART 1

Framing the problem: labels, the metric, the plateau and the sizing

Choosing a label and a metric for a marketplace, deciding what to do when a good model stops improving, and finding which resource binds.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Search ranking	Given a query and a set of candidate items, put them in order so that the item the user wants comes first. Here the items are listings and the wanted item is the one the guest books.
Impression	One listing shown to one guest in one position in the results of one search.
NDCG	Normalized discounted cumulative gain. Each position has a discount, so a relevant item near the top counts more. The gains are summed and divided by the best possible sum for that query.
Pairwise training	Training on pairs of examples from the same search, here one booked and one not booked, so that the model learns to score the booked one higher.
Plateau	A stage where a model that was improving through repeated experiments stops moving the target metric, so that A/B tests come back neutral.
A/B test	Showing different versions to randomly split groups of users at the same time and comparing a behavior metric, here bookings.
FLOP and multiply-add	A multiply-add is one multiplication plus one addition, counted as 2 floating point operations. A layer with a inputs and b outputs costs a times b multiply-adds per example.
Input bottleneck	A situation where the accelerator waits for data, because reading, parsing or copying the examples takes longer than computing on them.
Where to read	Haldar et al.: Applying Deep Learning to Airbnb Search (arXiv:1810.09591, KDD 2019)

01 What is the label, and who produces it?

FRAMING

SCENARIO

The paper trains a ranker for listing search. A colleague proposes using clicks on listing pages as the label because they are plentiful. The paper uses bookings. Explain the choice, and say exactly where the training examples come from.

HOW TO THINK ABOUT IT

Compare what a click and a booking say about a listing. Then read how the paper builds its pairs, and ask who is missing from the data.

Q What does the paper train on?

A Pairs of one booked listing and one not-booked listing, built from the listings seen by a guest who booked. The loss is cross-entropy on the score difference (source-confirmed).

- In offline evaluation the booked listing's impression has relevance 1 and every other impression 0.

Q Why a booking and not a click?

A A click shows momentary interest. A booking needs the guest to commit and the listing to match. The paper frames success as sessions that end with a booking (source-confirmed).

- The cost: bookings are rarer and arrive later. Most searches never end in a booking.

Q Who is missing from the data?

A Guests who did not book. A search session without a booking supplies no pair, which follows from the paper's wording that pairs come from the listings seen by a booker.

- The model learns what booking guests prefer. Browsers, who may differ, add nothing.

Q What else shapes the data?

A The paper says a new model trains on the logs of the current and previous production models. Those models decided which listings guests saw.

- The paper does not discuss the effect on the training data, so treat it as an open risk.

Q Reframe it.

A Pick the label that matches the outcome, then ask which users and which options ever produce it. A cleaner label can come from a narrower slice of behavior.

Basis: The pairs, the relevance labels and the log statement are from the paper (source-confirmed); the point about non-booking sessions is a reading of the paper's wording; the risk is reasoned.

02 The model has stopped improving. What now?

PLATEAU

SCENARIO

A tree-based ranker replaced a manual scoring function and gave one of the largest step improvements in bookings in the company's history. Years later, A/B tests keep coming back neutral. A director asks you to move to a neural network. Plan the move.

HOW TO THINK ABOUT IT

Use the paper's timeline and its own account of what went wrong first, and ask what each stage was meant to prove.

Q What does the paper say about the plateau?

A The gains in online bookings eventually saturated, with long spells of neutral experiments, which made the moment ripe for sweeping changes (source-confirmed).

- No single fault: the baseline had stopped responding to incremental work.

Q What went wrong at first?

A The team started with intricate custom architectures and was overwhelmed by their complexity. The first neural model that reached production was one hidden layer of 32 ReLU units, booking neutral against the trees (source-confirmed).

- Early on, all it saw was regression in offline metrics. Moving to deep learning, the paper says, is not a drop-in model replacement but a matter of scaling the system.

Q What was the first stage for?

A To prove that the whole neural pipeline was production ready and could serve live traffic, not to win. It used the same features and the same L2 loss as the trees.

- Keeping the inputs and the objective fixed isolates the pipeline as the thing being tested.

Q What was the schedule?

A Figure 2's labels: simple NN in 04/2017, LambdaRank NN in 06/2017, hybrid in 03/2018, deep NN in 06/2018 (source-confirmed).

- That is 14 months from the first neural launch to the deep model, with each stage launched and tested.

Q Reframe it.

A When a good model plateaus, change the system in stages. Make the first stage prove the plumbing with everything else held fixed, then change one thing at a time.

Basis: The plateau, the first model and the retrospective are from the paper (source-confirmed); the launch dates are the axis labels of Figure 2 (source-confirmed); the month count is arithmetic (recomputed).

03 Compute NDCG by hand

CALCULATION

SCENARIO

An interviewer says: a booked listing moves from position 2 to position 1 in one search, and another booked listing moves from position 10 to position 9 in another. Which improvement matters more to NDCG, and how much more? Use the standard discount of 1 divided by log2 of (position plus 1).

HOW TO THINK ABOUT IT

With one relevant item per search, NDCG is just the discount at its position. Compute the four discounts and subtract.

Q What are the discounts?

A Position 1: 1.000. Position 2: 1 ÷ log2(3) = 0.631. Position 9: 1 ÷ log2(10) = 0.301. Position 10: 1 ÷ log2(11) = 0.289.

- With one booked listing at relevance 1, the best possible total is 1, so NDCG equals the discount at the booked position.

Q What does each move gain?

A 2 to 1: 1.000 − 0.631 = **0.369**. 10 to 9: 0.301 − 0.289 = **0.012**.

- The first is about 31 times larger. The paper's own example is that moving from 2 to 1 gets priority over 10 to 9 (source-confirmed).

Q What does that imply for training?

A A loss that treats all mis-ordered pairs alike spends effort on the bottom of the list, where the metric barely moves. Weighting by the change in NDCG puts effort at the top.

- That is the idea behind LambdaRank (Question 10).

Q What can't you read from the paper?

A The size of the gains. Figure 2 shows relative gains in offline NDCG and online bookings as bars with no values (source-confirmed).

- Do not quote numbers for them. The text gives only about 1% offline for one regularization change.

Q Reframe it.

A Work a metric by hand on a tiny case before using it to drive training. It shows where the metric puts its weight.

Basis: The metric's definition and the 2-to-1 example are from the paper (source-confirmed); the discount formula is the standard one; arithmetic recomputed.

04 Where does the cost sit?

ESTIMATION

SCENARIO

The paper's final model takes 195 input features, then 127 and 83 ReLU units, trained on 1.7 billion pairs. It gives no search rate or inventory size. A manager asks whether the cost is in scoring, in training compute, or somewhere else. Use 2,000 candidates a query and a 4 KB static feature record per listing (both illustrative).

HOW TO THINK ABOUT IT

Count multiply-adds for scoring, then for one training pass, then bytes read for the same pass, and compare.

Q What does scoring cost?

A 195 × 127 + 127 × 83 + 83 = 24,765 + 10,541 + 83 = 35,389 multiply-adds, which is 70,778 FLOPs per candidate. For 2,000 candidates, 141.6 million FLOPs, about 0.14 GFLOP.

- A small amount of work per query. Scoring arithmetic is not the constraint.

Q What does a training pass cost in compute?

A A pair is two listings. With the backward pass at about 3 times the forward cost: 2 × 70,778 × 3 = 424,668 FLOPs. For 1.7 billion pairs, 7.2 × 10^14 FLOPs.

- At an assumed 5 TFLOP/s sustained, that is about 144 seconds.

Q What does it cost in bytes?

A If every example reads its listings' 4 KB of static features: 1.7 billion × 2 × 4 KB = 13.9 TB per pass. At 1 GB/s that is about 13,900 seconds, roughly 97 times the compute.

- The paper says the static features created an input bottleneck of multiple kilobytes per example, and that its GPU sat near 25% while time went into parsing CSV and copying data (source-confirmed).

Q What happens at ten times the data?

A Compute and bytes both grow ten times, so the ratio stays. The input path still limits training speed unless it is fixed (Questions 14 and 15).

- Scoring cost grows only with candidates per query and queries, not with training data.

Q Reframe it.

A Count every resource before optimizing one. A model with a tiny network can still be limited by how fast it is fed.

Basis: The architecture, the pair count, the input bottleneck and the GPU figure are from the paper (source-confirmed); the 2,000 candidates, 4 KB, 5 TFLOP/s and 1 GB/s are illustrative; arithmetic recomputed.

PART 2

Features and data: normalization, smoothness and the ID that failed

Why the same features that suit a tree model break a neural network, what a distribution plot reveals, and why a per-item embedding needs data per item.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Feature normalization	Rescaling a feature to a small range, here with the bulk between −1 and 1 and the median at 0. Trees only use the order of values. Networks also use the sizes.
z-score	(value − mean) ÷ standard deviation. It suits a feature whose distribution looks roughly normal.
Power-law feature and log transform	A feature with a long tail, such as a price. The paper uses log((1 + x) ÷ (1 + median)), which maps the median to 0 and compresses the tail.
Dying ReLU	A ReLU unit whose input is so negative that its output and gradient are zero for all examples, so it can no longer learn. Large inputs can cause large gradient steps that push units there.
High-cardinality categorical feature	A feature with very many values, such as a listing ID or a city and map-cell pair. It is turned into a learned embedding vector.
S2 cell	A tile of a hierarchical division of the globe. Level 12 is a fairly small tile, about neighborhood size.
Overfitting	A model that improves on its training data and not on new data, because it memorized details that do not generalize.
Multi-task learning	Training one model with several outputs and losses that share hidden layers, hoping that a plentiful signal helps a scarce one.
Where to read	Haldar et al.: Applying Deep Learning to Airbnb Search (arXiv:1810.09591, KDD 2019)

05 Why did the same features break the network?

NORMALIZATION

SCENARIO

The first neural attempt used exactly the features the tree model used. The paper says it went very badly: the loss saturated mid-training and more steps did nothing. Explain the cause and the fix, and show the transform on prices.

HOW TO THINK ABOUT IT

Ask what a tree uses from a number and what a network uses, then apply the paper's transform to a few prices.

Q What does a tree use from a feature?

A Only the order of values, as long as their relative ordering is meaningful. A price of 300 or 3,000 sits on the same side of a threshold (source-confirmed).

- So raw, wildly scaled features are fine for trees.

Q What does a network do with it?

A It multiplies by weights, so the size matters. Values outside the usual range cause large gradients, which can permanently shut off activations such as ReLU through vanishing gradients (source-confirmed).

- That explains a loss that stops moving.

Q What is the fix?

A Keep every feature in a small range, with the bulk in the interval from −1 to 1 and the median at 0. A roughly normal feature uses (x − mean) ÷ standard deviation. A power-law feature uses log((1 + x) ÷ (1 + median)).

- Take a median nightly price of 120 (illustrative).

Q Apply it to prices.

A Price 60 gives log(61 ÷ 121) = −0.685. Price 120 gives 0. Price 300 gives 0.911. Price 3,000 gives 3.211, where the raw value was 25 times the median.

- The tail is compressed, the median is at 0, and most listings land between −1 and 1.

Q Reframe it.

A A model family decides what properties your inputs need. Check what the model uses from each feature, and shape the data to that, instead of reusing inputs unchanged.

Basis: The failure, its mechanism, the two transforms and the target range are from the paper (source-confirmed); the median price of 120 is illustrative; arithmetic recomputed.

06 Plot the distribution first

DATA QUALITY

SCENARIO

The paper says it cares about smooth feature distributions, and gives three reasons. A teammate asks why a distribution plot is worth an engineer's time when range checks exist. Answer with the paper's examples.

HOW TO THINK ABOUT IT

For each example, ask what a range check would have said and what the plot showed.

Q What bugs did the plots catch?

A Prices logged in the wrong currency for some geographies, and, for stays over 28 days, a monthly price logged in place of the daily price. They appeared as spikes (source-confirmed).

- A monthly price of 3,600 in place of 120 a night would sit at log(3,601 ÷ 121) = 3.39 after the transform, which a range check on raw prices might accept.

Q What does smoothness do for generalization?

A The paper observes that hidden layers give progressively smoother distributions, and believes smooth inputs let upper layers interpolate for unseen values.

- Its sanity check scaled a test feature such as price to 2, 3 and 4 times and found NDCG remarkably stable (source-confirmed).

Q What did a lumpy feature reveal?

A Occupancy was not smooth. Listings have different minimum stays, up to months, so they fill at different rates. Adding average length of stay and normalizing occupancy by it gave a smooth shape.

- The lesson: a feature that is lumpy in one dimension may be smooth in a higher one, so ask whether the missing dimension is available to the model.

Q How was location handled?

A Raw latitude and longitude became the logarithm of the offset from the center of the map the user is viewing. It is a lossy many-to-one map and lets the model learn properties that depend on distance.

- Properties of specific places come from a separate categorical feature (Question 7).

Q Reframe it.
A A plot of the data finds problems no rule was written for. Look at each distribution, and ask what its odd shape says about the data or about a missing input.

Basis: The three reasons and the examples are from the paper (source-confirmed); the 3,600 example and its value are illustrative arithmetic.

07 Embed every listing ID?

FEATURES

SCENARIO

Neural networks make it easy to give every listing its own learned embedding, as recommenders do for videos. The paper tried it. Predict what happened, explain why, and give the arithmetic.

HOW TO THINK ABOUT IT

Ask how many training examples each ID has, and what limits that number for a listing compared with a video.

Q What happened?
A Listing IDs mostly led to overfitting. NDCG improved clearly on the training set and not on the test set (source-confirmed, Figure 5).

- The embedding memorized each listing's history.

Q Why does it work for videos and words?
A There is no limit to the amount of interaction an item can have. A popular video can get a huge number of views, so data per item is easy to get (source-confirmed).

- An embedding needs substantial data per item to converge.

Q What limits a listing?
A The physical world. Even the most popular listing can be booked at most 365 times in a year, and typical listings far fewer (source-confirmed).

- Over two years, the bound is 730 bookings. With a typical 30 a year (illustrative) a 32-dimensional embedding is fitted from about 60 positive examples.

Q What did the team do instead for location?
A It hashed the query's city together with the listing's level-12 S2 cell into one categorical feature, and learned an embedding for the pair. Many listings share a cell, so data per ID is large (source-confirmed).

- The old tree pipeline for neighborhood preference was heavily engineered and ignored the prices of booked listings.

Q Reframe it.
A An embedding per item needs data per item. Count the data each ID can ever have, and group IDs into units that have enough.

Basis: The failure, its explanation, the 365 bound and the S2 feature are from the paper (source-confirmed); the 30 bookings a year and the 32 dimensions are illustrative; arithmetic recomputed.

08 Review the price transform

CODE REVIEW

SCENARIO

A colleague writes this price feature for the neural ranker, following the paper's power-law transform. Offline NDCG is flat and the model ignores price. Premium and budget markets look alike to the model. Find the faults.

```
import math

def normalize_price(prices):
    # prices: nightly prices of the candidate listings for ONE query
    median = sorted(prices)[len(prices) // 2]
    return [math.log((1 + p) / (1 + median)) for p in prices]
```

HOW TO THINK ABOUT IT

Check what the median is taken over, whether serving can reproduce the training value, and what the paper says about odd prices.

Q What is the median taken over?

A The candidate prices of one query. In every search the middle price becomes 0, so a market where nights cost 900 and one where they cost 60 produce the same feature values.

- The model loses the absolute level of price, which is a main signal. The paper's transform uses the feature's median over the data, not per query.

Q Can serving reproduce it?

A It depends on which candidates happen to be in the set, so the same listing gets a different value from query to query. A training pipeline that sees different candidate sets gives a different feature than serving does.

- A fixed median, computed offline and shipped with the model, gives the same value everywhere.

Q What else is missing?

A Handling of bad prices. The paper found monthly prices logged for stays over 28 days and currency errors. Those become outliers that the transform keeps.

- Convert to a nightly price before the transform, and flag values far outside a plausible range.

Q State the fix.

A Compute the median once over a training sample and store it with the model. Convert monthly prices to nightly first, and clip extreme values after the transform.

- Add a test: the same listing and price give the same value in any query.

Q Reframe it.

A A statistic used to normalize a feature must be fixed, shared by training and serving, and computed over the right population. A per-request statistic erases the signal.

REFERENCE CODE

```
import math

PRICE_MEDIAN = 120.0        # computed once over a training sample, shipped with the model

def nightly_price(price, nights):
    return price / 30.0 if nights > 28 else price        # monthly rate logged for long stays

def normalize_price(price, nights):
    p = min(max(nightly_price(price, nights), 0.0), 20_000.0)
    return math.log((1 + p) / (1 + PRICE_MEDIAN))
```

Basis: The transform and the data errors are from the paper (source-confirmed); the 900 and 60 night markets are illustrative; the fix was checked by hand and not run.

PART 3

The models: trees, LambdaRank, the hybrid and the deep network

When to leave a tree model, what a pairwise weighted loss does, what the hybrid taught, and how to read a learning curve.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

GBDT	Gradient-boosted decision trees: a sum of many shallow trees, each trained to correct the errors of the ones before. Scoring follows each tree from root to leaf and adds the leaf values.
Pairwise loss	A loss on a pair of examples from the same query, here the cross-entropy of the score difference between the booked and the not-booked listing.
LambdaRank weight	A multiplier on each pair's loss equal to how much NDCG would change if the two listings swapped positions, so errors near the top count more.
LambdaMART	A boosted-tree model trained with the LambdaRank idea. It shows that trees can optimize a ranking metric too.
Factorization machine (FM)	A model that represents each listing and query as a vector, here 32-dimensional, and predicts a booking probability from how their vectors interact.
Leaf index feature	For a tree model, the number of the leaf a candidate ends in for each tree. Fed to a network as a categorical feature.
Learning curve and generalization gap	A plot of training and test performance against the amount of training data. The gap between them shows how much the model memorizes.
Technical debt	Complexity that was cheap to add and costs effort every time the system changes. A model that depends on other models is an example.
Where to read	Haldar et al.: Applying Deep Learning to Airbnb Search (arXiv:1810.09591, KDD 2019)

09 Trees or a neural network?

DECISION

SCENARIO

Your team has a tree ranker that works and has plateaued. The paper ends with a wholehearted yes to deep learning and also says trees are arguably at par for smaller problems. A colleague says that means the paper contradicts itself. Resolve it, and give a rule for your own team.

HOW TO THINK ABOUT IT

Separate what changed from model family to scale, and list what each side costs, using the paper's own words.

Q What is the paper's recommendation?

A A wholehearted yes, partly for strong online gains and partly because deep learning changed the roadmap. Confined to smaller scales, models like GBDT are arguably at par and easier to handle, and are still used for focused medium-sized problems (source-confirmed).

- The two statements are about different situations: scale against a focused problem.

Q What did the neural network add here?

A A model that kept improving as data grew ten times (Figure 4), and a shift of effort from hand-made feature math to the objective and to whether all users are represented.

- The paper says moving to deep learning is not a drop-in replacement but a matter of scaling the system.

Q What did it cost?

A Normalization discipline the trees did not need (Question 5), a rebuilt input pipeline (Question 14), a custom scoring library (Question 16), and much harder feature analysis (Question 18).

- The abstract says deep learning was steep learning for the team.

Q What is a rule for your team?

A Move when a simple model has plateaued, you have the data to scale, and you can afford to rebuild the surrounding system. Stay when the problem is focused and medium sized.

- Run the move in stages (Question 2) and judge each by an A/B test.

Q Reframe it.

A A model choice is a choice about the whole system around it. Match the family to the scale of the data and to what your team can operate.

Basis: The recommendation, the caveat and the abstract's remark are from the paper (source-confirmed); the rule for a team is reasoned.

10 What does LambdaRank do to the loss?

ALGORITHM

SCENARIO

The paper's breakthrough came from combining a neural network with the idea behind LambdaRank. A candidate says: that proves trees cannot be trained for NDCG. Explain what the loss does, compute an example, and challenge the claim.

HOW TO THINK ABOUT IT

Compute the weight and the loss for one pair, then ask what the paper tested.

Q What are the two changes?

A A pairwise form, with the loss on the score difference of a booked and a not-booked listing, and a weight on each pair equal to the change in NDCG from swapping the two positions (source-confirmed).

- The first simple NN used an L2 regression loss with booked = 1 and not booked = 0.

Q Compute one pair.

A Booked listing at rank 2, another at rank 1. Weight = |0.631 − 1.000| = 0.369. If the scores differ by 0.5, the base loss is log(1 + e^−0.5) = 0.474, so the weighted loss is 0.369 × 0.474 = **0.175**.

- For booked at rank 10 against rank 9 the weight is 0.012, so the same score gap costs 0.006.

Q Where do the ranks come from?

A In the paper's code they come from sorting the model's current scores within the sampled listings of a search, so the weights change as the model changes.

- The weights are treated as constants in the update, not as something to differentiate through.

Q Does the paper show trees cannot do this?

A No. The paper's tree model used L2 regression, and the paper cites a reference whose title includes LambdaMART, a tree model trained with the same idea. The paper did not test it.

- What it shows is that the pairwise, NDCG-weighted loss was the first breakthrough for the neural network, after a simple start that went nowhere much.

Q Reframe it.

A Do not credit a model family with something that was a change of objective. Ask whether the old family could have used the new loss, and say that you did not test it.

Basis: The loss and the weights are from the paper (source-confirmed); the LambdaMART point is from the paper's own references and its L2 statement; the worked loss is arithmetic (recomputed).

11 Combine three models, then drop two

ARCHITECTURE

SCENARIO

The team's other models, a boosted tree model and a factorization machine, matched the neural network on test data and still ranked different listings highly. They built a network that takes the tree leaves and the FM's prediction as inputs. Explain the reasoning, the cost, and what happened next.

HOW TO THINK ABOUT IT

Ask what it means when equally good models disagree, and what you must run to keep the combination alive.

Q Why combine them?

A The paper says the three models had comparable performance on test data, but the listings each one up-ranked were quite different. That suggests each captured signal the others missed (source-confirmed).

- Equal averages with different rankings is a sign of complementary errors.

Q How were they combined?

A The FM's final prediction went in as a feature. From the tree model, the index of the leaf activated in each tree went in as a categorical feature. The FM maps listings and queries to a 32-dimensional space.

- The network learned how to weigh them.

Q What did it cost?

A Three models to train, version and serve, and each one's inputs and failures to track. The paper calls the complexity staggering and says some of the issues of a well-known technical-debt paper began to appear.

- Every change to a tree or the FM changes the inputs of the network.

Q What replaced it?

A The final leap deprecated all that complexity by scaling the training data ten times and moving to a network with two hidden layers (source-confirmed).

- The hybrid was a stage that earned the right to a simpler design, not the end state.

Q Reframe it.

A When models disagree, they hold different information. Use that as a clue and as a way station, and look for the simpler design that captures it, which here was more data.

Basis: The motivation, the inputs, the cost statement and the replacement are from the paper (source-confirmed); the point about maintenance burden is reasoned.

12 Review the LambdaRank loss

CODE REVIEW

SCENARIO

A colleague writes this pairwise loss for one search, with the booked listing at index 0, and says it implements the paper. The loss is sometimes infinite or not a number, and the model does not improve at the top of the list. Find the faults.

```
import numpy as np

def lambdarank_loss(logits):
    # logits: (n,) scores for one search; logits[0] is the booked listing
    order = np.argsort(-logits)                 # listing indices, best first
    discount = 1.0 / np.log2(order + 1)
    weights = np.abs(discount[0] - discount[1:])
    diff = logits[0] - logits[1:]
    return np.mean(weights * np.log1p(np.exp(-diff)))
```

HOW TO THINK ABOUT IT

Check what argsort returns, what the discount is at rank 1, and whether the weight should carry a gradient.

Q What does argsort give here?

A The listing indices in order of score, best first. It is not the rank of each listing. Indexing the discount by listing index mixes the two.

- The rank of each listing is argsort of argsort, plus 1 for a 1-based rank.

Q What goes wrong at the top?

A The discount is 1 divided by log2 of (order + 1). When order is 0, log2(1) = 0 and the division is infinite, so the weight is infinite for the top-scored listing.

- With 1-based ranks the discount at rank 1 is 1 ÷ log2(2) = 1, as the NDCG example in Question 3 shows.

Q What about gradients?

A The weights are computed from the logits. In a framework that differentiates, the weights would receive gradients and distort learning. The paper computes them outside the graph, as constants.

- Detach the weights, or compute them from copies of the scores.

Q State the fix, and a test.

A Compute 1-based ranks with argsort of argsort of the negative scores, derive the discounts from them, make the weights constants, and take the weighted pairwise loss.

- Test: with the booked listing already first, the loss is small. With two listings and a booked one at rank 2, the weight is 0.369 (Question 10).

Q Reframe it.

A Check each index and each constant in a metric-based loss on a case small enough to compute by hand. Off-by-one errors in ranks are silent until a zero appears.

REFERENCE CODE

```
import numpy as np

def lambdarank_loss(logits):
    ranks = np.argsort(np.argsort(-logits)) + 1       # 1-based rank of each listing
    discount = 1.0 / np.log2(ranks + 1)
    weights = np.abs(discount[0] - discount[1:])      # constants: no gradient
    diff = logits[0] - logits[1:]
    return np.mean(weights * np.log1p(np.exp(-diff)))
```

Basis: The loss is from the paper (source-confirmed); the faults are standard numerical and indexing errors; the fix was checked by hand against the Question 3 and 10 numbers and not run.

13 Read a learning curve

EVALUATION

SCENARIO

The paper plots NDCG on training and test data against the number of training pairs, and says that training on 1.7 billion pairs closed the generalization gap. A manager asks what that means and whether to buy more data or a bigger model. Explain.

HOW TO THINK ABOUT IT

Describe what each curve does as data grows, and compare with the listing ID experiment, which showed a different picture.

Q What does a learning curve show?

A Training and test performance as the data grows. With little data, training performance is high and test is lower, because the model memorizes. As data grows the two converge (source-confirmed, Figure 4).

- Closing the gap means the model's training performance now predicts its test performance.

Q What did the listing ID experiment look like?

A A significant improvement in NDCG on the training set and none on the test set (Figure 5): the gap stayed wide and more data was not available per ID.

- Two curves, two stories: one where more data fixed the gap and one where it could not.

Q What does this say about data and model size?

A When the gap is wide and data can grow, add data. When the gap is closed, a bigger model may help. The paper scaled the data by ten times and used two hidden layers.

- It does not report the effect of each change separately, so you cannot attribute the gain.

Q What did more data cost?

A Reading far more examples through the input path. The pipeline rebuild (Question 14) is what allowed training data to grow from weeks to months.

- Data size is limited by how fast it can be fed.

Q What does the paper leave open?

A Whether the deep model could have been launched directly, skipping the stages. It asks and does not answer outright.

- Say so, and avoid claiming that the stages were necessary or wasted.

Q Reframe it.

A Use learning curves to decide between more data and a bigger model. The shape of the gap tells which one the problem needs.

Basis: The learning curve, the 1.7 billion pairs, the listing ID plot and the open question are from the paper (source-confirmed); the advice on data against model size is reasoned.

PART 4

Serving, evaluation and operations

Feeding a GPU, what a static embedding is for, scoring in another language, trusting offline numbers, and understanding what the model uses.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Protobuf and Dataset pipeline	Writing training examples as compact binary Protobuf records and reading them with TensorFlow's Dataset API, instead of parsing CSV text and copying it through feed_dict.
GPU utilization	The share of time the GPU is busy computing. A low value means it is waiting for data.
Static (quasi-static) feature	A listing property that rarely changes, such as location, bedrooms, amenities and guest rules.
Interop overhead	The cost of converting data between two language runtimes, for example from Java to a library written in another language, on every call.
Offline and online evaluation	Offline: scoring a model on logged data. Online: measuring real guests in an A/B test.
Permutation importance	Shuffling one feature's values across examples and measuring how much performance drops. It assumes features are independent.
Ablation	Removing a feature, retraining and measuring the change in performance.
Position bias and selection bias	Position bias: a listing shown first is booked more because it was first. Selection bias: logs only record what earlier models chose to show.
Where to read	Haldar et al.: Applying Deep Learning to Airbnb Search (arXiv:1810.09591, KDD 2019)

14 Feed the GPU

TRAINING PIPELINE

SCENARIO

Training reads CSV files and copies the values through feed_dict. GPU utilization is near 25%. The paper retooled the pipeline with Protobufs and the Dataset API and reports a 17 times speedup with utilization near 90%. A manager asks whether raising utilization from 25% to 90% explains the 17 times.

HOW TO THINK ABOUT IT

Compute the ratio of the two utilizations, then ask what else could account for the rest.

Q What is the ratio?

A 90 ÷ 25 = 3.6 times. If utilization were the whole story, the speedup would be 3.6 times, not 17.

- Put the other way, a 17 times speedup ending at 90% would have started at 90 ÷ 17 = 5.3%, not 25%.

Q So what else changed?

A The work itself. Parsing text and copying through feed_dict cost time in the step itself, and compact binary records read by a pipeline that overlaps loading and computing cut it.

- The paper does not split the 17 times between the two. Do not derive one number from the other.

Q What did it enable?

A Scaling the training data from weeks to months, which the paper says allowed it to attack the problem with far more data (source-confirmed).

- The pipeline paid for the 10 times data in the deep model (Question 13).

Q How would you find the bottleneck yourself?

A Measure the time per step with the real input and with data already in memory. If they differ by a lot, the input path is the limit.

- The paper's own finding was GPU use near 25% with time spent parsing and copying.

Q Reframe it.

A A headline speedup bundles several changes. Check that the numbers you quote can be derived from each other before treating them as one cause.

Basis: The 17 times, the utilization figures and the weeks-to-months statement are from the paper (source-confirmed); the 3.6 and 5.3 are arithmetic (recomputed); the diagnosis method is reasoned.

15 A static embedding: for training, not serving

PIPELINE

SCENARIO

Many listing features rarely change: location, bedrooms, amenities and guest rules. The paper packs them into a non-trainable embedding indexed by listing ID and keeps it in GPU memory. A colleague draws it as a serving cache with write-through updates. Explain what it is and what it costs.

HOW TO THINK ABOUT IT

Read where in the paper it appears and what problem it solves, then size it.

Q Where does the paper put it, and why?

A In the system engineering section, to remove an input bottleneck. Reading all these features with every training example meant multiple kilobytes per example loaded from disk through the CPU (source-confirmed).

- The embedding eliminated that traffic, and made it possible to explore models that use details of tens of listings a user interacted with.

Q How big is it?

A It depends on the number of listings, not on the number of examples. With an assumed 2 million listings at 4 KB each, 8.2 GB.

- The per-example alternative reads 13.9 TB per pass for 1.7 billion pairs, so each listing's record is read about 1,700 times.

Q What does it cost?

A For features that changed during the training period, the embedding holds one value. The paper says this trades off a small level of noise against training speed (source-confirmed).

- A training example sees the listing as of the snapshot, not as of the search.

Q Is it a serving cache?

A The paper does not say. It does not describe how the table is built, how a new or changed listing reaches it, or how serving gets these values.

- Treat any serving design as your own. A mismatch between the snapshot used in training and the live values used in serving is a risk to test for.

Q Reframe it.

A Read where a technique appears and what problem it was introduced for before reusing it. A trick for the training path is not a statement about serving.

Basis: The embedding, its purpose, the noise trade-off and the missing serving detail are from the paper (source-confirmed); the 2 million listings and 4 KB are illustrative; arithmetic recomputed.

16 Score a TensorFlow model inside a Java server

SERVING

SCENARIO

Search runs in a Java server that does retrieval and scoring, and the models are trained in TensorFlow. The paper found no efficient way to score the models in Java and wrote its own library. A reviewer asks whether that was wise. Weigh it.

HOW TO THINK ABOUT IT

Size what crossing the language boundary would move per query, then list what a custom library takes on.

Q What does a call across languages move?

A 2,000 candidates × 195 features × 4 bytes = 1.56 MB per query, to convert and copy in and out (illustrative sizes).

- At 1,000 queries a second on one server, 1.56 GB a second of copying and conversion, before any model arithmetic.

Q What does the paper say?

A When shipping TensorFlow models to production in early 2017 it found no efficient way to score them in a Java stack. Converting data back and forth between Java and another language added latency that was a blocker (source-confirmed).

- It created a custom neural network scoring library in Java to adhere to the strict latency requirement of search, and expects to revisit the choice.

Q What does the custom library take on?

A Reimplementing every layer and activation the models use, keeping the numbers identical to the trained model's, and updating the library whenever the architecture changes.

- Add a parity test that compares library scores with the training framework's on a fixed sample of listings.

Q What does the paper not say?

A A latency budget or a measured latency, so no figure can be quoted. Compare candidates with your own budget.

- Alternatives such as a model server called over the network have their own call cost. Measure it against the budget.

Q Reframe it.

A Count the data that crosses a boundary on every request. A small model can still be too slow if each call copies megabytes between runtimes.

Basis: The finding, the custom library and the strict latency requirement are from the paper (source-confirmed); the 2,000 candidates, 195 four-byte features and the 1,000 queries a second are illustrative; arithmetic recomputed; the parity test is reasoned.

17 Trust the offline number?

EVALUATION

SCENARIO

A change to the regularization raises offline NDCG by about 1%. Your team wants to ship it. The paper has this exact case. Say what to do, and what else the paper tried.

HOW TO THINK ABOUT IT

Recall what happened in the paper with this change and with the long-view model, and read the hyperparameter results for what they say about effort.

Q What happened with the 1%?

A The paper replaced dropout with hand-crafted noise shapes that took each feature's distribution into account, and got about 1% offline NDCG. It failed to get a statistically significant online improvement (source-confirmed).

- An offline gain was not a gain in bookings.

Q What else behaved that way?

A The long-view model raised long views by a large margin and left bookings neutral. Long views correlate with bookings and have a large orthogonal component, for example high-priced listings or unusual ones.

- Both are cases of an easier-to-move measure that did not transfer to the metric that counts.

Q What did the paper do about dropout?

A It tried several flavors, and all led to a slight degradation in offline metrics. Its reading: the randomness produced invalid scenarios that distracted the model, unlike data augmentation that mimics valid ones.

- Other settings: Xavier initialization, uniform embeddings, Adam with default settings in a lazy variant, a fixed batch size of 200. A broad search gave confidence in these choices and no meaningful gains.

Q What should the team do?

A Run the change in an A/B test on bookings and decide on that. Use the offline number to choose what to test.

- The paper's rule: a new model is tested online for a statistically significant increase in conversions.

Q Reframe it.

A An offline gain is a reason to test, not a reason to ship. Ask whether the thing that moved is the thing the business counts.

Basis: The 1% result, the long-view result, the dropout findings and the settings are from the paper (source-confirmed); the advice is reasoned.

18 Which features matter, and what is not covered?

ANALYSIS

SCENARIO

The paper says estimating feature importance was where the team took a step back with neural networks. It tried four methods. Explain why three of them misled, what the fourth showed, and what the paper leaves out.

HOW TO THINK ABOUT IT

For each method, find the assumption it makes about the model or the features and where the assumption fails.

Q Why did score decomposition and ablation mislead?

A Decomposition tried to split the final score among input nodes, which has no clean answer across a non-linear activation. Ablation removed one feature and retrained, and the change was about the noise of retraining, because the model made up the loss from redundant features.

- With ten near-copies of one signal, dropping any one shows no loss, and dropping all ten shows a large one. That is the Ship-of-Theseus problem the paper names.

Q Why did permutation mislead?

A Shuffling one feature assumes it is independent of the others. Number of rooms came out as one of the most important features, because shuffling it produced listings that never exist, such as one room with a high price for many guests.

- It was still useful to find features that did not matter: if shuffling changed nothing, the model probably did not depend on the feature.

Q What did TopBot show?

A It ranks the listings of each test query with the model and compares feature distributions of the top-ranked and bottom-ranked ones, with nothing perturbed. Top-ranked prices skewed lower, showing price sensitivity. Review distributions looked alike, so the model was not using reviews as expected.

- A finding to investigate, not a proof, since the paper treats it as a direction for further work.

Q What does the paper not discuss?

A Position bias, where a listing shown first is booked because it was first, and selection bias from training on logs of earlier models. The positional discount in NDCG belongs to the metric and is not a correction for either.

- Say so rather than assuming it was handled.

Q Reframe it.

A Every importance method assumes something about the model or the data. Use several, expect them to disagree, and check the assumption of each before believing it.

Basis: The four methods, their outcomes and the TopBot findings are from the paper (source-confirmed); the absence of a bias discussion is a statement about the paper's text; the ten-copies example is illustrative.

Sources and confidence

The case study this book is built on is the AI System Design Primer's Capstone #4. The one source is the Airbnb paper (arXiv:1810.09591, published at KDD 2019), whose full text was read on 2026-10-17. Its results are mostly unlabeled bar charts, and its text gives few numbers, so every concrete size in this book is an illustrative assumption. It says nothing about retrieval, the latency budget or how a changed listing reaches the static embedding, so answers about those are design arguments made for this book and were not run on real systems.

- Haldar, Abdool, Ramanathan, Xu, Yang, Duan, Zhang, Barrow-Williams, Turnbull, Collins, Legrand: Applying Deep Learning to Airbnb Search, arXiv:1810.09591 (KDD 2019), full text read 2026-10-17
https://arxiv.org/abs/1810.09591